

Breaking Changes Since 1.7.2

This is the breaking-change record for projects updating from Easy Idle Game 1.7.2 to 2.0. It covers source compatibility, serialized-data migration, save compatibility, and behavior changes. [Migration Guide](#) (EasyIdleGame > md > MigrationGuide.md) is the practical companion record.

Before Updating

1. Commit or back up the project and a representative production save.
2. Search custom scripts for every symbol in [Removed And Renamed Symbols](#).
3. Import the update, let Unity finish serialization, and inspect representative `Business` , `Reward` , `Achievement` , `PrestigeManager` , and `BusinessMergeRecipe` assets.
4. Replace legacy string groups with typed group assets before relying on targeted effects.
5. Test purchase prices, production, randomized rewards, offline progress, save/load, location travel, shop completion, achievements, and prestige.

For each feature demo below, open the `.unity` scene and select the `ReadMe.asset` beside it for setup notes and the resource prefix.

Source/API Migration Table

Old API or contract	Current API or contract	Required
<code>UpgradeType.cost</code>	<code>UpgradeType.purchaseCost</code>	Rename source refere enum value remains 2 contextual cost types upgrades, levels, and
<code>IValidatable</code> and <code>Business.GetValidationErrors()</code>	Inspector validation with <code>ConditionalCommentArea</code> or custom editor validation	Remove interface imp Put runtime validation invalid data must be r Inspector.
<code>Business.outputs</code> and <code>IProducible.Outputs</code> as the authored production model	<code>Business.outputTables</code> and <code>IProducible.OutputTables</code>	Old outputs migrate t <code>DropStrategy.All</code> table directly and choose a <code>Outputs</code> is now a flatte view.
<code>Reward.rewards</code> , <code>Achievement.rewards</code> , <code>PrestigeManager.prestigeRewards</code>	<code>rewardTables</code> and <code>prestigeRewardTables</code>	Old lists migrate into Code that previews re table average/expecte
<code>IProducible.ApplyOuputs(...)</code>	<code>ApplyOutputs(BigNumber)</code>	Correct the spelling a returned <code>List<Output></code> needs actual grants.
<code>IProducible.GetActualProduction(...)</code> used as a preview	<code>GetOptimisticActualProduction(...)</code>	Use optimistic APIs fo <code>GetActualProduction()</code> current production an
<code>DailyRewardsManager.ClaimDailyRewards(int)</code>	<code>TryClaimDailyRewards(int, out List<Output>)</code>	Handle <code>false</code> for unav claimed days and disp grants.
<code>PrestigeManager.PrestigeGame()</code>	<code>TryPrestigeGame(out outputs)</code> or the overload returning <code>ResetSummary</code>	Check success. Use ou and <code>ResetSummary</code> whe

		must react to what was reset/preserved.
<code>BusinessHolder.StartProduction()</code> as the primary entry point	<code>TryStartProduction(),</code> <code>TryProduceManually(...),</code> Or <code>StartProduction(amount, out round, context)</code>	Prefer the Try method because all copies are production inputs are
<code>BusinessHolder.producing</code> and mutable <code>productionTime</code>	<code>ActiveProductions, CurrentlyProducing,</code> <code>GetProductionProgress()</code>	Treat the old properties as compatibility shims. Do not use concurrent-round state
<code>BusinessHolder.OnProductionRoundFinished()</code> override/call	<code>OnProductionRoundFinished(ProductionRound)</code>	Preserve the round passed in. Do not roll outputs and are authoritative result.
<code>BusinessHolder.ClaimOutputs()</code> returning void	<code>ClaimOutputs(object context = null)</code> returning <code>List<Output></code>	Use the returned list for popups, analytics, or to
<code>BoostsManager.IsAutoProduceOverrideActive(string group)</code>	<code>IsAutoProduceOverrideActive(Business)</code>	Pass the business so that membership and wrap are evaluated.
String <code>businessGroup</code> , <code>boost group</code> , and <code>upgrade group</code> authoring	<code>BusinessGroup</code> assets and typed target lists	Legacy strings remain for produce warnings. Create both source members
<code>BusinessMergeRecipe.input / output</code> holder arrays	<code>inputs</code> and <code>outputTables</code>	Serialized arrays migrate. Do not consume businesses/containers. Any supported output
<code>TryMergeBusinesses(recipe)</code> returning only bool	<code>TryMergeBusinesses(recipe, out List<Output>)</code>	Handle granted outputs. USE <code>GetModifiedMergeInputs</code> , <code>GetEffectiveMergeDuration</code> , <code>GetEffectiveMergeCoolDown</code>
<code>ForceMergeBusinesses(recipe)</code> returning bool	<code>ForceMergeBusinesses(recipe)</code> returning <code>List<Output></code>	Only force after project consumes inputs and outputs.
Runtime-created recipes appearing automatically	<code>CachedResources.LoadAll<BusinessMergeRecipe>("")</code> lookup	Put discoverable recipes in call <code>BusinessesManager.Refresh</code> after creating/replacing
<code>BusinessesManager.GetBusinessAmount, AddBusinesses, IsBusinessUnlocked, GetCurrentItemLevel</code>	<code>iBuyable.GetItemAmount,</code> <code>ForceBuyItemsForFree,</code> <code>iUnlocked.IsItemUnlocked,</code> <code>iLevelupable.GetCurrentItemLevel</code>	Update convenience interfaces/containers
<code>BusinessesManager.GetTotalProductionPerSecondOfType(string)</code>	Typed <code>GetTotalProductionPerSecond(Currency, Business, ...)</code>	Pass a Currency OR Business outputs by display name
Nullable-style <code>GetTotalProductionPerSecond(Currency?, Business?, ...)</code>	Reference parameters <code>Currency</code> and <code>Business</code> (either may be null)	Remove nullable values. overrides/wrappers.
<code>UpgradesManager.GetUpgradeAmount / old upgrade unlock helper</code>	<code>iBuyable.GetItemAmount(upgrade) / iUnlocked.IsItemUnlocked(upgrade)</code>	Use shared manager interfaces

<code>UpgradeHolder.OnBought(BigNumber)</code>	<code>ApplyBoughtAmount(BigNumber, SourceType)</code>	Timed acquisition and separated. Call normal APIs where possible.
Holder <code>AddAmount(...)</code> methods without a source	Overloads accepting <code>SourceType</code>	Pass <code>Purchase</code> , <code>Advertisement</code> , an appropriate source upgrades, audio, and context. Defaults preserve compatibility.
Custom <code>IBuyable</code> implementations	Add <code>UseGeometricBoughtAmount</code> and <code>BuySound</code>	Return <code>false/null</code> when whether free grants still use geometric prices.
Custom <code>IBuyableHolder</code> implementations	Add <code>BoughtAmount</code> , <code>OnBought</code> , <code>OnAdded</code> , and <code>Pay_BuyEvent</code>	Track current, lifetime, amounts separately; fit into the normal buy flow.
Custom <code>IUnlockable</code> implementations	Add <code>UpperLocks</code> and <code>UnlockSound</code>	Return <code>null/empty</code> when state now requires locked conditions.
Custom <code>IUnlockableHolder</code> / manager implementations	Add <code>onUnlocked</code> , holder/item events, holder enumeration, and auto-unlock checks	Wire events if project depends on unlock tracking
Custom <code>ILevelupable</code> / holder implementations	Add level-up audio/events and source-aware upgrade methods	Forward the <code>SourceType</code> callbacks.
Custom <code>IProducible</code> implementations	Add output tables, partial-cost/consume flags, round mode, production locks, upper production locks, and production audio	Implement real/optimized second calculation pass
Custom <code>IProducibleHolder</code> implementations	Add active rounds, current amount, manual production, start/finish events, and returned claims	A single Boolean/time represents the contract

Important Behavior Changes

Purchases, totals, caps, and sources

Holders now distinguish:

- `Amount` : current inventory or owned amount.
- `TotalAmount` : lifetime amount acquired, including grants.
- `BoughtAmount` : amount acquired through purchase flows.

When `UseGeometricBoughtAmount` is enabled, geometric price progression uses `BoughtAmount` ; free rewards no longer increase future prices. Additions are clamped by item and typed-group capacity. `SourceType` is propagated through additions and purchases for contextual upgrades, player statistics, events, and audio.

Workaround for an intentional uncapped/debug grant: use an explicit project debug path and restore invariants afterward; normal `ForceBuyItemsForFree` still respects holder capacity in several systems and should not be assumed to mean “ignore every cap.”

Try: `EasyIdleGame > Demos > FeatureDemos > LocksAndBuyAmounts > LocksAndBuyAmounts.unity` .

Production rounds, costs, and output calculation

Production is round based:

- `SingleRoundLiveAmount` folds eligible newly added copies into the active round.
- `SingleRoundSnapshot` leaves new copies for the next round.
- `ConcurrentRounds` starts independent rounds for idle copies.

Outputs are rolled at round start and applied at finish. Production locks and upper locks can block starts.

`allowPartialProductionCost` reduces a requested amount to the affordable amount; otherwise the whole start fails.

`consumeOnProduce` removes the producing copies after completion. Preview, current-per-second, offline, and actual-round calculations now have distinct contexts.

Custom production behavior should use `ProductionOutputModifierAsset` plus `ProductionModifierManager`, not mutate shared `DropTable` assets during a round. Modifier order matters; `IOrderedProductionOutputModifier.Priority` controls it.

Try: `ProductionAndCollection`, `ProductionCosts`, `BusinessChaining`, and `OfflineProfitApplication`.

Random rewards and drop evaluation

`DropStrategy.All` evaluates every output; `WeightedRandom` chooses from weighted candidates. `dropChance`, `weight`, and `outputMaxAmount` can be modified contextually. `EvaluateOnce` rolls once then scales the amount; `EvaluatePerInstance` evaluates independent instances and automatically falls back to expected-value calculation for large batches.

Workaround for large repeated grants: use `EvaluateOnce` when independent per-instance randomness is not a design requirement; use `EvaluatePerInstance` when normalized batch behavior is desired.

Try: `DropTableRewards`.

Contextual upgrades

`UpgradeType.cost` became `purchaseCost`; additional types now target duration, chance, weight, production-output scaling, cooldown, production inputs, upgrade purchases, level-up costs, and merge inputs. `UpgradeBlock` may use multiply/add operations and `UpgradeBlockFilter` may scope by source, target assets/groups, levels, manual/offline state, and merge recipe. `OverrideUpgradeModifierBlock` can alter override values.

Old code that only calls global multiplier aggregation will not expose contextual effects in custom previews. Use `UpgradeEffectResolver` and pass the correct context.

Try: `BoostsAndUpgrades`, `ProductionCosts`, `MergingRecipes`, and `TimedUpgradeQueues`.

Typed groups and group capacity

`BusinessGroup`, `CurrencyGroup`, `BoostGroup`, `UpgradeGroup`, `ManagerGroup`, `LocationGroup`, `AchievementGroup`, and `ShopItemGroup` provide typed membership. Business/currency groups can impose shared caps; business/currency/upgrade groups can be lock targets; shop-item groups filter offer pools; and prestige exclusions support every listed family except shop items. Legacy business-group strings are fallback-only.

Try: `BusinessGroupTargeting`, `LocksAndBuyAmounts`, and `PrestigeResets`.

Achievements are tiered

Achievements can now be claimed multiple times through `claimAmount`; requirements scale by `claimMultiplier` per claimed tier. `AchievementHolder` tracks the current claim tier. Code that treated “claimed once” as terminal should instead check the holder’s claimed amount/remaining tiers.

Try: `PlayerProgression`.

Prestige reset scope

Prestige can reset locations and achievements, preserve explicit assets or entire typed groups, accept per-call additions through `PrestigeResetOptions`, and return `ResetSummary`. Project-specific world/board cleanup should run around `TryPrestigeGame` or `OnPrestige`; do not duplicate the package reset of holders.

Try: `PrestigeResets` .

Profit and time skip

`ProfitCalculator.ApplyProfit` and `CalculateAndApplyProfit` return `ProfitApplicationSummary` and invoke `OnProfitApplied` . The applied result may be lower than `ProfitData` because of currency/business/group caps, production stockpiles, and active-round adjustments. UI should show `ProfitData` as calculated earnings and use the summary when it must report what was actually accepted.

Try: `OfflineProfitApplication` and `OffersAndTimeSkip` .

Shop and external completion

Real-money and ad attempts only delegate through events. The integration must call `CompleteExternalPurchase(item, source)` after verified success. Calling completion before verification grants rewards. Offers select eligible, weighted `ShopItem` s and are not a receipt-validation layer.

Try: `ShopRewardCompletion` and `OffersAndTimeSkip` .

Save-worthy event debounce

Economy managers now call `EconomyChangedEvents.NotifySaveWorthyStateChanged` . Notifications are debounced within a frame and expose source/suppression information. Autosave listeners must avoid saving recursively while loading; guard with an `_isLoading` flag.

Behavior corrections

These defects are now fixed; projects that worked around them should review the new behavior:

- `SaveAndLoad.Path` uses `Path.Combine` and trims leading separators from `savePath` . Saves at the old concatenated location are migrated automatically on `Awake()` / `Load()` , and `IrreversiblyDeleteSave()` deletes both locations. Remove project-side path overrides that only compensated for the missing separator.
- `CompleteExternalPurchase(item, amount, source)` floors the requested amount, rejects non-positive requests, and clamps the grant to the remaining `maxPurchases` ; the reward count always matches the amount added. `TryConsumeItem` / `TryConsumeAmount` reject non-positive amounts.
- `TryClaimDailyRewards` rejects non-positive, future, and rewardless days without recording them; repeating entries with a non-positive `level` are skipped; applied outputs carry the `RecurringReward` source instead of `Generic` . Remove duplicate project-side claim validation if it exists, or keep it — both are safe.
- `ProfitCalculator.CalculateProfit` returns an empty `ProfitData` for zero or negative durations.
- Merge cooldown follows contextual `UpgradeType.cooldown` blocks (multiplicative — values below 1 shorten it) instead of the merge speed multiplier. Rebalance any merge cooldown tuning that relied on `UpgradeType.speed` .
- `Lock.IsUnlocked` and `IsUpperUnlocked` validate every configured manager dependency before evaluating requirement values. A missing manager now throws an `InvalidOperationException` naming the requirement and manager instead of satisfying the requirement or producing an inconsistent null-reference failure.

Serialized Asset Migrations

Unity deserialization migrates these legacy fields:

- Business `outputs` to `outputTables` .
- Reward, level reward, achievement, daily reward, and prestige reward lists to guaranteed tables.
- Merge recipe input/output holder arrays to `inputs` and `outputTables` .
- Legacy name/icon/model/description fields into `Metadata` across supported `ScriptableObject`s.
- `UpgradeType.cost` remains safe in assets because `purchaseCost` keeps enum value `2` .

Migration is compatibility code, not proof the authored result is correct. Inspect output strategies, chances, weights, min/max amounts, metadata, group assignments, upper locks, and empty tables after import. Save assets once verified so they no longer depend on hidden legacy fields.

Business wrappers

`BusinessWrapper.SyncData()` copies serialized data from `underlyingBusiness`, rewrites self-references in costs, production costs, upgrade costs, lower/upper locks, output tables, and level rewards, then applies wrapper scales. It runs on enable/validation and can overwrite edits to copied fields.

Workaround for intentionally divergent behavior: make a standalone `Business`, or put supported differences in wrapper metadata/scales instead of editing copied fields.

Save Compatibility

Default saves now include locations, shop items, achievements, custom stats, `TotalAmount`, `BoughtAmount`, active boosts, and expanded daily-reward state. Missing total/bought values are seeded from older fields during load.

Custom `SaveFile` subclasses and `SaveAndLoad` components must:

- Call or preserve `base.Load()` behavior.
- Include custom fields without dropping new base fields.
- Guard autosave callbacks during load.
- Clamp offline elapsed time with `maxOfflineSeconds` (and any idle-cap override) before calculating profit.
- Test an untouched pre-update save and a save produced after migration.

Removed And Renamed Symbols

Search project scripts for:

```
UpgradeType.cost
IValidatable
GetValidationErrors
ApplyOutputs
GetActualProduction(
ClaimDailyRewards
PrestigeGame(
GetBusinessAmount
AddBusinesses
IsBusinessUnlocked
GetCurrentItemLevel
GetTotalProductionPerSecondOfType
GetUpgradeAmount
IsBoostUnlocked
BoostsManager.IsAutoProduceOverrideActive(
UpgradeHolder.OnBought
BusinessMergeRecipe.input
BusinessMergeRecipe.output
BusinessMergeRecipeHolder
producing
productionTime
outputs
rewards
```

Some names still exist as obsolete or serialization shims. Treat search hits as review points rather than blind replacements: `outputs` / `rewards`, for example, may be legitimate local variables.

Serialized Enum Stability And Current Deprecations

Unity serializes `UpgradeType` and `OverrideUpgradeType` as integer values in authored assets. Do not reorder or insert members into the existing numeric range. Project-specific members should use stable explicit values starting at `1000`.

Current source retains these obsolete compatibility categories:

- Flattened business/producible output access.
- Legacy group strings and string multiplier overloads.
- Single-production fields and methods on `BusinessHolder`.
- Old direct production-time/output/cost helpers.
- `LevelReward`; use `Reward`.
- Included list displays; prefer Unity layout components for new list layout code.

Version 1.6 changed business/player level-reward storage incompatibly, and 1.7.2 removed some obsolete functions. Keep compatibility fields until every supported asset and save has passed migration tests; do not assume warning-only APIs will remain indefinitely.

Migration Verification Matrix

Area	Scene
Core buy/produce/upgrade loop	EasyIdleGame > Demos > FeatureDemos > 01_BasicFlow > BasicFlow.unity
Costs, bought amount, caps, lower/upper locks	EasyIdleGame > Demos > FeatureDemos > LocksAndBuyAmounts > LocksAndBuyAmounts.unity
Actual/manual/concurrent production and claims	EasyIdleGame > Demos > FeatureDemos > ProductionAndCollection > ProductionAndCollection.unity
Input-limited production	EasyIdleGame > Demos > FeatureDemos > ProductionCosts > ProductionCosts.unity
Random/weighted rewards	EasyIdleGame > Demos > FeatureDemos > DropTableRewards > DropTableRewards.unity
Typed targeting/contextual effects	EasyIdleGame > Demos > FeatureDemos > BoostsAndUpgrades > BoostsAndUpgrades.unity
Recipe migration	EasyIdleGame > Demos > FeatureDemos > MergingRecipes > MergingRecipes.unity
Save/offline/cap application	EasyIdleGame > Demos > FeatureDemos > OfflineProfitApplication > OfflineProfitApplication.unity
Locations, shop, achievements, daily, prestige	LocationTravel, ShopRewardCompletion, PlayerProgression, DailyRewards, PrestigeResets under FeatureDemos

Recommended Migration Order

1. Resolve Unity/package compatibility and compile enum/API renames.
2. Update custom interface implementations.
3. Inspect and resave migrated ScriptableObjects.
4. Replace string groups and validate shared capacities.
5. Update callers to handle success flags, output lists, summaries, and source types.
6. Update custom preview code to use contextual resolvers and optimistic APIs.
7. Load a copy of an old production save and verify totals/bought amounts.
8. Run the closest feature demos and project integration tests.
9. Consult the maintained Markdown scripting reference and current source for the current API surface.